

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра №806 «Вычислительная математика и программирование»

Курсовая работа по курсу
“Дискретный анализ”

Пространственный поиск

Студент: Немкова Анастасия Романовна

Группа: М8О-308Б-22

Преподаватель: Макаров Никита Константинович

Оценка: _____

Дата: _____

Подпись: _____

Москва, 2024

Содержание

1. Репозиторий
2. Постановка задачи
3. Метод решения
4. Описание алгоритма
5. Описание программы
6. Тест производительности
7. Выводы

Репозиторий

https://github.com/anastasia-nemkova/DA_labs

Постановка задачи

Реализуйте систему для определения принадлежности точки одному из многоугольников на плоскости.

Формат ввода

В первой строке дано число n - количество многоугольников. Далее для каждого из n многоугольников дано число m - количество вершин и в следующих m строках заданы координаты вершин x, y . Далее задано число q - количество точек. В следующих q строках даны пары чисел x, y - координаты точек.

Формат вывода

Для каждого запроса выведите номер многоугольника, внутри которого содержится точка (многоугольники нумеруются с нуля) либо -1.

Метод решения

Задача заключается в определении, принадлежит ли точка какому-либо из многоугольников на плоскости. Для решения этой задачи было принято использование метода *ray casting* и *r-tree* для ускорения поиска подходящих многоугольников.

R-дерево - структура данных, предназначенная для индексирования многомерных данных, таких как координаты геометрических объектов. В основе R-дерева лежит иерархическая структура, где каждый узел содержит несколько прямоугольников MBR. Листовые узлы содержат MBR для объектов (в данном случае многоугольников), а внутренние узлы — для их групп.

Построение R-дерева

Начинаем с пустого дерева. Для каждого многоугольника вычисляем его MBR и добавляем в дерево. На каждом уровне дерева объекты группируются в узлы, причем каждый узел ограничен минимальным ограничивающим прямоугольником (MBR). Когда узел заполняется, его нужно разделить. Таким образом, R-дерево строится динамически: каждый новый многоугольник добавляется в дерево, и если узел переполняется, он разбивается, а родительский узел обновляется.

В худшем случае время построения R-дерева составляет $O(n \cdot \log(n))$, где n — количество многоугольников, поскольку для каждого объекта вычисляется его MBR и происходит вставка в дерево.

Вставка в R-дерево

При вставке нового многоугольника в дерево, мы сначала находим подходящий листовой узел, куда можно добавить новый элемент. Для этого ищем узел, чьи границы

будут минимально увеличены для включения нового MBR. Если узел переполняется (количество объектов в узле превышает максимальную допустимую величину), происходит разбиение этого узла, и часть его объектов передаются в новый узел, который добавляется на тот же уровень дерева. В случае переполнения узла на более высоком уровне, процесс повторяется, и дерево может увеличиваться на один уровень.

Время вставки одного объекта в R-дерево составляет $O(\log(n))$.

Поиск в R-дереве

Поиск начинается с корня дерева. Для каждого узла проверяется пересечение его MBR с многоугольником:

- Если MBR полностью внутри многоугольника, все его дочерние элементы добавляются в результаты.
- Если MBR пересекается с многоугольником, рекурсивно проверяются дочерние элементы данного узла.

Время поиска одного объекта в R-дереве также составляет $O(\log(n))$, поскольку мы проходим только по тем узлам, чьи MBR могут пересекаться с точкой.

Таким образом, мы находим все возможные MBR многоугольников, в которых может находиться точка, и далее для более точной проверки принадлежности точки многоугольнику используем ray casting для каждого из найденных кандидатов.

MBR(Minimum Bounding Rectangle) - прямоугольник, который охватывает все вершины многоугольника. Он определяется минимальными и максимальными значениями координат по осям x и y . Основное преимущество заключается в том, что он позволяет быстро определить, может ли точка попасть в многоугольник (если точка не лежит внутри MBR, она точно не лежит в многоугольнике).

Ray Casting - алгоритм, использующийся для определения, лежит ли точка внутри многоугольника. Алгоритм заключается в том, что из точки бросается луч в одном направлении (например, вправо) и считаются пересечения с ребрами многоугольника. Если пересечений нечетное количество, точка внутри многоугольника, если четное — снаружи.

Время работы для многоугольника с m вершинами составляет $O(m)$, так как необходимо проверить каждое ребро.

Описание алгоритма

Мы начинаем с того, что считываем входные данные. В первой строке у нас есть количество многоугольников. Для каждого многоугольника мы считываем его вершины. Сначала мы считываем количество вершин многоугольника, а затем сами вершины. После этого считываем количество точек, для которых нужно проверить принадлежность многоугольнику, и сами эти точки.

Далее мы создаем структуры данных. Для представления точек, которые будем использовать, мы создаем структуру Point, которая содержит два поля — координаты x и y .

Для представления многоугольников создаем структуру Polygon. В этой структуре мы храним уникальный идентификатор многоугольника и список его вершин.

Чтобы эффективно искать, какие многоугольники могут содержать точки, мы используем структуру данных — R-дерево. Оно использует принцип "разбиения" данных на меньшие прямоугольные области MBR. Каждый узел дерева хранит такие прямоугольники, которые охватывают собой одну или несколько объектов, и для каждого прямоугольника хранится ссылка на объект.

Каждый многоугольник будет представлен своим минимальным ограничивающим прямоугольником (MBR). Для каждой точки мы будем искать подходящие прямоугольники в дереве и проверять, содержится ли точка внутри этих прямоугольников.

Когда мы добавляем многоугольник в дерево, мы сначала рассчитываем его минимальный ограничивающий прямоугольник (MBR). Мы выбираем подходящий лист в дереве, куда нужно добавить этот прямоугольник. Если в листе становится слишком много прямоугольников, то мы выполняем разбиение листа, чтобы сохранить структуру дерева. При разбиении листа мы выбираем "лучшую" часть для каждого из новых узлов, чтобы минимизировать площадь охвата.

Для каждого запроса на точку, мы ищем в R-дереве, какие прямоугольники могут содержать эту точку. Для этого мы начинаем с корня дерева и рекурсивно проверяем, какие дочерние узлы могут содержать точку. Если прямоугольник в узле дерева не пересекается с точкой, мы его пропускаем.

Если прямоугольник пересекает точку, то мы проверяем, содержится ли точка внутри этого прямоугольника. Если да, мы добавляем этот прямоугольник в список кандидатов, а затем проверяем, относится ли эта точка непосредственно к многоугольнику.

Для каждого многоугольника, в который попала точка, мы выполняем проверку на принадлежность точки самому многоугольнику. Для этого используется метод *gaucasting*. Также нужно учитывать случай, когда точка лежит на ребре многоугольника. В этом случае мы проверяем, лежит ли точка на одном из ребер многоугольника с точностью до некоторой погрешности.

После того как мы нашли многоугольники, содержащие точку, выводим их идентификаторы. Если таких многоугольников несколько, выводим все их идентификаторы. Если точка не лежит внутри ни одного многоугольника, выводим -1.

Описание программы

Программа содержит структуры Point, которая хранит информацию о каждой точке: координаты x и y ; MBR - представляет минимальный ограничивающий прямоугольник многоугольников, он включает методы:

- contains - проверка, содержится ли точка внутри MBR.
- expand - расширение текущего MBR, чтобы он охватывал также другой MBR.
- overlaps - проверка, пересекаются ли два MBR.
- area - вычисление площади MBR.

Структура Polygon - представляет многоугольник, который состоит из нескольких точек (вершин), содержит методы:

- getMBR - получение MBR для многоугольника.
- isOnEdge - проверка, лежит ли точка на ребре многоугольника.
- rayCasting - проверка, находится ли точка внутри многоугольника.

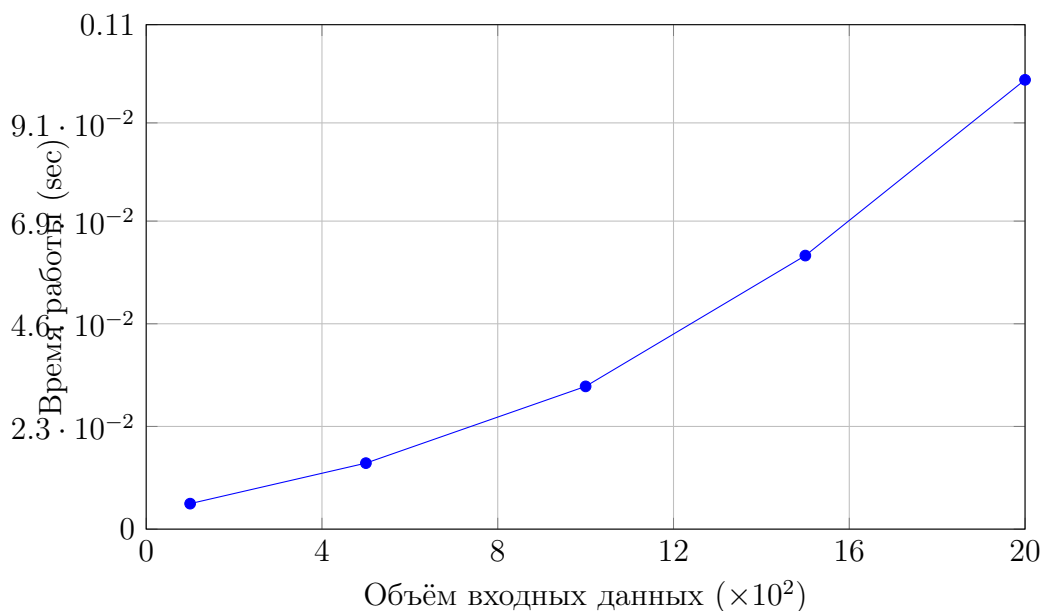
Также в программе реализован класс R-дерева, который используется для организации и поиска геометрических объектов. Он содержит методы:

- insert - вставка MBR и соответствующего индекса многоугольника в R-дерево.
- search - поиск точек, содержащихся в многоугольниках, с помощью рекурсивного обхода дерева.
- chooseLeaf - выбор подходящего листа для вставки.
- linearSplit - деление узла на два, если количество элементов превышает максимально допустимое.
- adjustTree - корректировка дерева после вставки нового элемента, если необходимо.
- findParent - поиск родительского узла для корректировки дерева.
- calculateMBR - вычисление минимального ограничивающего прямоугольника для узла дерева.
- searchRecursive - рекурсивный поиск по дереву.

Тест производительности

Для измерения производительности создадим тесты с различными размерами входных данных. Подсчет времени будем производить с помощью библиотеки `chrono`, которая позволяет фиксировать время в начале и конце работы программы. Для каждого теста будем задавать размеры n - количества многоугольников: для первого теста 100, для второго 500, для третьего 1000, для четвертого 1500 и для пятого 2000, с фиксированным размером количества точек $m = 100$ и количеством вершин от 3 до 32

На графике представлена зависимость времени выполнения поиска многоугольников, которым принадлежит точка от объема входных данных



Время поиска в дереве составляет $O(\log(n))$ для одной точки, где n — количество объектов в R-дереве. Проверка точек на принадлежность многоугольникам - $O(n * k)$ для всех многоугольников.

Таким образом, если обрабатывать q точек для поиска, общая временная сложность будет:

В худшем случае: $O(q * (\log(n) + m * k))$, где: q — количество точек для поиска. n — количество многоугольников. m - количество найденный кандидатов, которым может принадлежать точка k — количество вершин в многоугольнике.

Выводы

В ходе данной курсовой работы были изучены методы пространственного поиска данных с использованием различных структур данных, таких как R-дерево, которое эффективно решает задачи поиска пересечений и ближайших объектов в многомерных пространствах. Реализован алгоритм вставки, поиска и обработки запросов для R-дерева, а также рассмотрены особенности его временной и пространственной сложности.

Также исследованы альтернативные подходы, такие как использование пространственных индексов для ускорения поиска. Особое внимание было уделено оптимизации работы с большими объемами данных и снижению времени обработки запросов. В дальнейшем можно было бы реализовать персистентное декартово дерево для пространственного поиска, что позволило бы хранить и восстанавливать историю изменений дерева, улучшив работу с временными данными.